

```

import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler # For normalizing data
import tensorflow as tf
from tensorflow import keras

# Load the dataset from CSV file
df = pd.read_csv('1_boston_housing.csv')
print(df.head()) # Check first 5 rows #

# Separate features (X) and target price (y)
X = df.drop('MEDV', axis=1).values # All columns except price
y = df['MEDV'].values # 'medv' = median house value #

# Split into training (80%) and testing (20%) sets
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42) #

# Normalize features — neural networks work better with scaled data
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train) # Fit on train, transform train
X_test = scaler.transform(X_test) # Only transform test (no fit!) #

# Build the neural network model
model = keras.Sequential([
    keras.layers.Dense(64, activation='relu', # First hidden layer, 64 neurons
        input_shape=(X_train.shape[1],)),
    keras.layers.Dense(32, activation='relu'), # Second hidden layer, 32 neurons
    keras.layers.Dense(1) # Output layer — 1 value (price)
]) #

# Compile model with loss function and optimizer
model.compile(
    optimizer='adam', # Adam optimizer adjusts learning rate
    loss='mse', # MSE = Mean Squared Error (regression loss)
    metrics=['mae'] # MAE = Mean Absolute Error (readable metric)
) #

# Train the model for 100 epochs
history = model.fit(
    X_train, y_train,
    epochs=100, # 100 passes over the training data
    batch_size=32, # Process 32 samples at a time
    validation_split=0.1, # 10% of train data for validation
    verbose=1 # Print progress
) #

# Evaluate model on test data
mse, mae = model.evaluate(X_test, y_test, verbose=0)
print(f"Test MSE: {mse:.2f}")
print(f"Test MAE: {mae:.2f}") # Lower MAE = better predictions #

# Predict on first 5 test samples
predictions = model.predict(X_test[:5])
print("Predicted prices:", predictions.flatten())

```

```
print("Actual prices:", y_test[:5]) #
```